The Quaternion Julia Set, in Clojure

2012-09-18 13:35:06

Original: `https://nicknash.me/2012/09/18/quaternion-julia-clojure/`

Graphics programming is one of my favourite things to do when learning a new programming language. Since I have been learning a bit of Clojure recently, I decided to see if anyone had written up a quaternion Julia set renderer. I couldn't find one after a bit of googling, so here we are.

My goal here isn't to put together the julia set renderer to end them all, but just some Clojure code that produces interesting output while still being pretty simple. Hopefully it'll be interesting to other people learning Clojure.

Of course, there are implementations out there in plenty of other languages, especially GPU targetted. Some of them produce very beautiful **real-time** renders, see for example Iñigo Quilez's neat article (`http://iquilezles.org/www/articles/juliasets3d/juliasets3d.htm`) on 3D julia sets rendered on a GPU. I originally rendered the quaternion Julia set in C++/OpenGL some years back after seeing Paul Bourke's POVRay renders (`http://paulbourke.net/fractals/quatjulia/`). I've also seen a nice description (`http://nakkaya.com/2009/10/04/fractals-in-clojure-buddhabrot-fractal/`) of the Buddhabrot drawn using Clojure.

1 The Julia Set

So what are we talking about? We want to draw some pictures of the quaternion Julia set. A point, z_0 is in the Julia set if the sequence defined by

$z_{n+1} = z_n^2 + c$

doesn't tend to infinity. In our case, the points are quaternions and we just iterate the above until $|z_n|$ gets too large, or we hit an upper bound on the maximum number of iterations and give up on trying to make the series diverge.

This is what that looks like in Clojure:

```
(defn outside-julia? [q c max-iterations]
  (loop [iter 0 z q]
    (if (> (vec-length z) 4)
      true
      (if (< iter max-iterations)
        (recur (inc iter) (vec-add (quat-mul z z) c))
        false))))
```

This code uses 4 as the magic length at which we decide the series is diverging. We'll take a look at implenting the functions `vec-length`, `vec-add` and `quat-mul` next.

2 Vectors and Quaternions

Like any demoscener or hobbiest graphics programmer, I've implemented linear algebra libraries many times. Compared to C++ or Java, Clojure is just beautiful. I wouldn't like to know how much slower it is, but, compared to say C++ it's amazing that these one-liners give the generality of n-dimensional vector operations so clearly and concisely.

```
(defn vec-add [& args] (vec (apply map + args)))
(defn vec-sub [& args] (vec (apply map - args)))
(defn vec-dot [u v] (reduce + (map * u v)))
(defn vec-length [v] (Math/sqrt (vec-dot v v)))
(defn vec-scale [r v] (vec (map #(* r %) v)))
(defn vec-normalize [v] (vec-scale (/ 1 (vec-length v)) v))
```

We can re-use these functions on quaternions, but we do need to define quaternion multiplication, for which we'll need the cross product:

```
(defn det-2x2 [a b c d] (- (* a d) (* b c)))
(defn vec-cross-3d [u v]
  (vector (det-2x2 (u 1) (u 2) (v 1) (v 2))
          (det-2x2 (u 2) (u 0) (v 2) (v 0))
          (det-2x2 (u 0) (u 1) (v 0) (v 1))))
(defn quat-mul [q1 q2]
  (let [r1 (first q1) v1 (vec (rest q1))
        r2 (first q2) v2 (vec (rest q2))]
    (cons (- (* r1 r2) (vec-dot v1 v2))
          (vec-add (vec-scale r1 v2)
                   (vec-scale r2 v1)
                   (vec-cross-3d v1 v2)))))
```

Now that we have the tools to generate terms in the julia set sequence, we can start thinking about how to draw it.

3 Naive Ray Marching

One easy way to take a look at a 3D slice of the quaternion julia set is just to follow parallel rays through each pixel on the screen-area we're drawing to. To do this for each pixel at screen position (x', y') we pick some minimum z and consider a series of quaternions

$(x, y, z, p), (x, y, z + \Delta, p), (x, y, z + 2\Delta, p), \ldots$

At each quaternion, we use the `outside-julia?` function shown above to see if we're inside or outside the set.

Here Δ is the amount we march along the ray between checking if we're inside or outside the set. The parameter p just controls where we're taking the 3D slice from, since the set is 4D after all.

After marching along in Δ-sized steps if we find that we're inside, we refine the estimate of the intersection point by marching in the opposite direction along the ray in smaller increments, checking

the quaternions beginning at the first z-coordinate, z_{in} we found to be inside the set, until we find one outside again:

$(x, y, z_{in}, p), (x, y, z_{in} - \delta, p), (x, y, z_{in} - 2\delta, p), \ldots$

The Clojure implementation of this forward and then backward ray-marching looks like this:

```
(defn ray-march-z [keep-marching? get-next-z initial-z min-z max-z num-steps]
  (let [dz (float (/ (- max-z min-z) num-steps))]
    (loop [z initial-z]
      (if (keep-marching? z)
        (recur (get-next-z z dz)) z))))

(defn ray-march-forward-then-backward [outside? min-z max-z
               num-coarse-steps num-fine-steps]
  (let [z (ray-march-z #(and (outside? %) (< % max-z))
                       #(+ %1 %2)
                       min-z min-z max-z num-coarse-steps)]
    (if (< z max-z) (ray-march-z #(and (not (outside? %)) (> % min-z))
                                 #(- %1 %2)
                                 z min-z max-z num-fine-steps) max-z)))
```

After ray-marching a bunch of pixels, we'll still only have a buffer of their z-coordinates, and we still need to render them somehow. We can do that by defining a simple lighting model.

4 Lighting

Now that we have our buffer of z-coordinates (z-buffer), we need to colour it somehow. I'll take a very simple approach here, just using the good-old Blinn-Phong lighting model.

Before we can do that though, we'll need surface normals - or some approximation to them. We'll use the so-called central differencing technique: compute vectors between neighbouring pixels in the z-buffer and take their cross product. The implementation looks like this:

```
(defn get-buf-vector [buf-pos buf-pos-to-viewport buf-getter size]
  (let [before (clamp-range (- buf-pos 1) size)
        after (clamp-range (+ buf-pos 1) size)
        vp-before (buf-pos-to-viewport before)
        vp-after (buf-pos-to-viewport after)
        buf-before (buf-getter before)
        buf-after (buf-getter after)]
        [(- vp-before vp-after) (- buf-before buf-after)]))

(defn get-normal
  "Compute a normal to the z-buffer at buf-x, buf-y by differencing neighbours"
                [buf-x buf-y
                 x-buffer-to-viewport y-buffer-to-viewport
                 zbuffer size]
  (let [u (get-buf-vector buf-x x-buffer-to-viewport #(aget zbuffer % buf-y) size)
        v (get-buf-vector buf-y y-buffer-to-viewport #(aget zbuffer buf-x %) size)]
      (vec-normalize (vec-cross-3d (vector (u 0) 0 (v 1)) (vector 0 (v 0) (v 1))))))
```

As for the Blinn-Phong lighting itself, it's just a single function that returns the light intensity given the surface normal, view direction, light direction and material properties.

```
(defn zero-clamp [v] (if (< v 0) 0 v))

(defn phong-light [normal view-dir light-dir diffuse specular shininess]
  (let [half-vec (vec-scale 0.5 (vec-add view-dir light-dir))
        n-dot-l (vec-dot normal light-dir)
        clamped-diffuse (zero-clamp n-dot-l)
        h-dot-v (vec-dot half-vec view-dir)
        clamped-specular (zero-clamp h-dot-v)
        spec-exp (Math/pow clamped-specular shininess)]
    (vec-add (vec-scale clamped-diffuse diffuse) (vec-scale spec-exp specular))))
```

We can now march some rays and switch on the lights. When we do, we'll get something like this:

The thing about our render is that, although it's kind of pretty, and definitely interesting, it's a bit rough looking. We could try and hack around to fix this. One obvious idea is to try and smooth the surface normals, with a Gaussian blur or something.

If we think about it a little, the real problem isn't the normals though, it's the way we've done our ray-marching. At least part of the problem is that we don't always find the first intersection, sometimes we'll step too far while marching forwards, and then march back to the wrong exterior point. Luckily, computer graphics people have been rendering these types of fractals since the '80s and there is a different approach to naive ray marching that has nicer properties and is much more efficient.

5 Distance Estimation

Instead of naively stepping along the rays to find intersection points, it turns out to be simple to compute a so-called *unbounding volume* from any point in space, that is, a sphere that is guaranteed not to contain any of the fractal. The details of this can be found in the original paper (`http://graphics.cs.illinois.edu/node/58`). IQ has also written a few nice articles (`http://iquilezles.org/www/index.htm`) about this.

With unbounding volumes, we compute a series of distances of decreasing size, which we use to march along our rays. A minimum distance is defined at which we decide to stop marching. This minimum distance, combined with guaranteeing never to over-step the mark and find an intersection that isn't the closest gives a much smoother look to the renders. They're also much quicker to generate. Here's an example of a render using this technique:

Missing diagram.
Source: `http://nicknash.me/wp-content/uploads/2012/09/distance-estimate-final`.jpg

Link: `http://nicknash.me/wp-content/uploads/2012/09/distance-estimate-final.jpg`

The Clojure for doing ray-marching with distance estimation is very simple:

```
(defn julia-distance-estimate [q c max-iterations]
  (loop [iter 0 dz [1.0 0 0 0] z q]
    (let [z-length (vec-length z)]
      (if (and (<= z-length 4) (< iter max-iterations))
        (recur (inc iter)
               (vec-add (vec-scale 2.0 (quat-mul z dz)) [1.0 0 0 0])
               (vec-add (quat-mul z z) c))
```

```
      (/ (* z-length (Math/log z-length)) (vec-length dz))))))

(defn unbounding-ray-march [c max-julia-iterations param max-distance-iterations
                            epsilon min-z vp-x vp-y]
  (loop [iter 0 z min-z dist (+ 1 epsilon)]
    (if (and (< iter max-distance-iterations) (> dist epsilon))
      (let [ dist-estimate (julia-distance-estimate [vp-x vp-y z param] c max-julia-
          iterations)]
        (recur (inc iter) (+ z dist-estimate) dist-estimate)) z)))
```

6 Code

The snippets above are collected together at my Github:

https://github.com/nicknash/quat_julia (`https://github.com/nicknash/quat_julia`)